

Practical 05: ERP data analysis for Decision Making

```
In [1]: 1 # Credits: this notebook was written by Ilaria Gavetti and Nin Khodorivsko for the Cognitive Neuroscien
2
3 # MNE-toolbox was used as reference to develop and implement the code.
```

```
In [2]: 1 import mne
2 import os
3 import numpy as np
4 import csv
5 import pandas as pd
```

Welcome to the data-analysis part of the Executive Functions practical. At this point, you should have:

1. collected your data
2. received an output csv file from the data collection procedure, ...

Follow the instructions below to properly upload and conduct your data-analysis:

Step 0: upload the files

First, I suggest creating a "base" directory, called "Executive functions practical".

Then, please upload all the eeg_data.npy file from your laptop in the directory `"../Decision Making practical/1. Raw files"`, where the `"..."` part is your personal path to get to **"Executive functions practical"** and **1. Raw files** is a folder in this directory which you should manually add. Also, upload the annotations.npy file in the **2. Loading & Digitization folder**.

Since since we will be saving intermediate files during the analysis, add the following folders to your **"Decision Making practical"** directory:

2. Loading & Digitization

3. ICA

4. Epoch (with inside 1 other folder called **Annotated**)

The code you will run already has these directories implemented.

Last, but not least, change your current working directory to **"Decision Making practical"**, to ensure the code runs smoothly.

Step 1: Loading

```
In [3]: 1 os.getcwd()

'C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_06\Decision Making practical'
```

```
In [4]: 1 data = np.load("1. Raw files/eeg_data.npy") # insert the eeg_data.npy file name instead of the "..."
```

```
In [5]: 1 data.shape

(126274, 18)
```

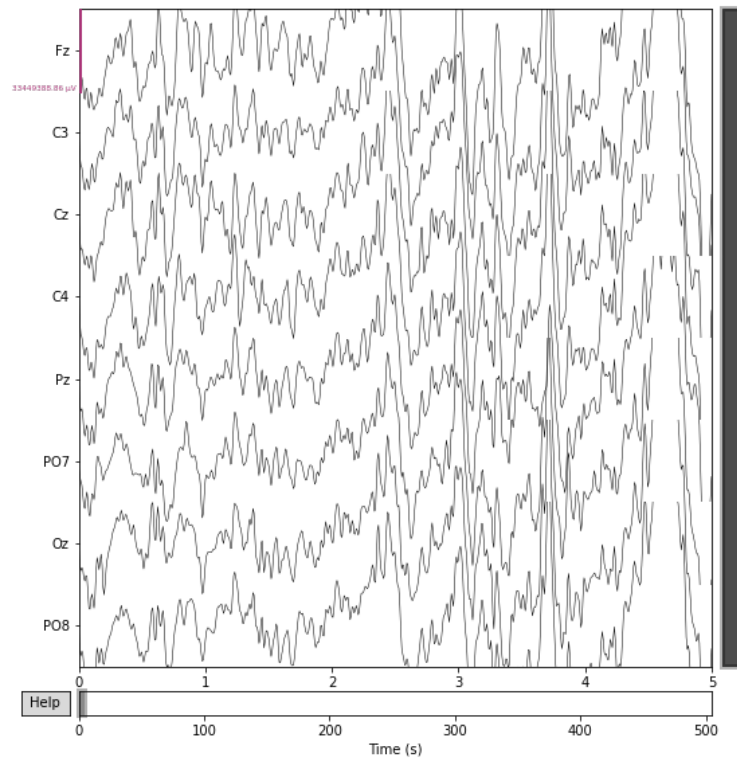
```
In [6]: 1 # Read the CSV file as a NumPy array
2
3 for file in os.listdir("1. Raw files"):
4     if file != '.ipynb_checkpoints':
5         if "eeg" in file:
6             print(file)
7             columns=['Time', 'FZ', 'C3', 'CZ', 'C4', 'PZ', 'P07', 'OZ', 'P08', 'AccX', 'AccY', 'AccZ', 'Gyro']
8             data = pd.DataFrame(data=data, columns=columns)
9             data_sub = data[["FZ", "C3", "CZ", "C4", "PZ", "P07", "OZ", "P08"]] # only select the elec
10            # Print file name
11            print("****"+file+"****")
12        elif "Unicorn" in file:
13            print(file)
14            data = pd.read_csv("1. Raw files/"+file, delimiter=",", verbose = True)
15            data_sub = data[["EEG 1", "EEG 2", "EEG 3", "EEG 4", "EEG 5", "EEG 6", "EEG 7", "EEG 8"]] #
16            # Print file name
17            print("****"+file+"****")
18
19        # Some information about the channels
20        ch_names_new = ["Fz", "C3", "Cz", "C4", "Pz", "P07", "Oz", "P08"] # from Unicorn BCI manual
21
22        # Sampling rate of the Unicorn BCI
23        sfreq = 250 # Hz
24
25        # Create the info structure needed by MNE
26        info = mne.create_info(ch_names_new, sfreq, ch_types = ["eeg"]*len(ch_names_new), verbose = Tru
27
28        # Set montage
29        info.set_montage(montage = 'standard_1020')
30
31
32        # Finally, create the Raw object
33        raw = mne.io.RawArray(data_sub.T, info, verbose = True)
34
35        # Filter
36        raw.filter(1,30)
37
38        # Plot it!
39        raw.plot(scalings="auto", duration = 5)
40        print("End")
41
42        # Save it!
43        raw.save("2. Loading & Digitization/"+file[0:8]+"_raw.fif", overwrite = True)
44
eeg_data.npy
****eeg_data.npy****
Creating RawArray with float64 data, n_channels=8, n_times=126274
Range : 0 ... 126273 = 0.000 ... 505.092 secs
Ready.
Filtering raw data in 1 contiguous segment
Setting up band-pass filter from 1 - 30 Hz

FIR filter parameters
-----
Designing a one-pass, zero-phase, non-causal bandpass filter:
- Windowed time-domain design (firwin) method
- Hamming window with 0.0194 passband ripple and 53 dB stopband attenuation
- Lower passband edge: 1.00
- Lower transition bandwidth: 1.00 Hz (-6 dB cutoff frequency: 0.50 Hz)
- Upper passband edge: 30.00 Hz
- Upper transition bandwidth: 7.50 Hz (-6 dB cutoff frequency: 33.75 Hz)
- Filter length: 825 samples (3.300 sec)

[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 0.0s remaining: 0.0s
```

```
[Parallel(n_jobs=1)]: Done 2 out of 2 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 3 out of 3 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 4 out of 4 | elapsed: 0.0s remaining: 0.0s
[Parallel(n_jobs=1)]: Done 8 out of 8 | elapsed: 0.0s finished
```

Using matplotlib as 2D backend.



End

Overwriting existing file.

Writing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_06\Decision Making practical\2. Loading & Digitization\eg_data_raw.fif

Closing C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_06\Decision Making practical\2. Loading & Digitization\eg_data_raw.fif

[done]

Step 2: Adding annotations manually

```
In [7]: 1 from datetime import datetime, timezone
        2 from datetime import timedelta
```

```
In [8]: 1 os.getcwd()
```

'C:\\Users\\IamAh\\Desktop\\UNIVERSITY - CSAI\\Cognitive Neuroscience\\Practical_06\\Decision Making practical'

Add the name of the annotations file instead of '...' in all the cells below

```
In [9]: 1 np.load("2. Loading & Digitization/annotations_1715160116.4333339.npy") # insert the annotations .npy f

array([[ 'timeout', '1715160120.4343064'],
       [ 'Possible win', '1715160141.4749591'],
       [ 'Possible win', '1715160162.4857464'],
       [ 'Possible loss', '1715160183.495436'],
       [ 'Possible loss', '1715160204.5059268'],
       [ 'Possible loss', '1715160225.517151'],
       [ 'Possible win', '1715160246.5257611'],
       [ 'Possible win', '1715160267.53793'],
       [ 'Possible loss', '1715160288.5484204'],
       [ 'Possible win', '1715160309.5585675'],
       [ 'Possible win', '1715160330.567997'],
       [ 'Possible win', '1715160351.5803807'],
       [ 'Possible loss', '1715160372.588301'],
       [ 'Possible win', '1715160393.5991414'],
       [ 'Possible loss', '1715160414.6085894'],
       [ 'Possible loss', '1715160435.6188457'],
       [ 'Possible loss', '1715160456.628219'],
       [ 'Possible loss', '1715160477.6382344'],
       [ 'Possible win', '1715160498.6496146'],
       [ 'Possible win', '1715160519.66158'],
       [ 'Possible win', '1715160540.675027'],
       [ 'Possible loss', '1715160561.6856246'],
       [ 'Possible win', '1715160582.6961558'],
       [ 'Possible loss', '1715160603.7095616']], dtype='<U32')
```

Up until now, the raw data does not contain any markers, meaning that each file is a continuous flow of EEG data without any marker indicating where the **possible win / loss** trials happen.

Nevertheless, we do know at what time from the recording onset each trial happens, so we need to make sure we annotate the EEG data with markers that reflect the information given in the **array**.

In the next cell, we make sure that the EEG data is annotated such that the program will know at what point of the continuous EEG data each trial happens, and how long each trial lasts until the onset of a new trial.

Annotating will allow us to "Epoch" the data, which will make our life easier later in the notebook!

```
In [10]: 1 files = os.listdir("2. Loading & Digitization")
```

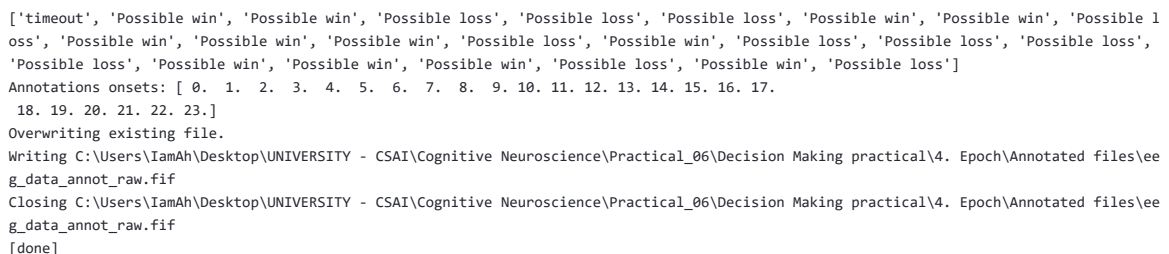
```
In [11]: 1 for file in files:
2         if file != '.ipynb_checkpoints_raw.fif' and "raw" in file:
3             sample_data_raw_file = "2. Loading & Digitization/"+file
4             raw = mne.io.read_raw_fif(sample_data_raw_file, preload=True)
5
6             raw.plot(scalings = "auto", duration = 5)
7
8             # Annotation
9             array = np.load("2. Loading & Digitization/annotations_1715160116.4333339.npy") # insert the an
10
11             description = []
12             onset = []
13             start = float(0)
14
15             for i in range(array.shape[0]):
16                 description.append(array[i][0])
17                 onset.append(start)
18
19                 # Increase
20                 start = start + 1 # we decide that each trial lasts one second
21
22             later_annot = mne.Annotations(onset=onset, duration=[1]*24, description=description) # we have
23
24             raw.set_annotations(later_annot)
25             print(description)
26             print("Annotations onsets:", raw.annotations.onset)
27
28
29             raw.save("4. Epoch/Annotated files/"+file[0:8]+"_annot_raw.fif", overwrite = True)
```

Opening raw data file 2. Loading & Digitization/eeg_data_raw.fif...

Range : 0 ... 126273 = 0.000 ... 505.092 secs

Ready.

Reading 0 ... 126273 = 0.000 ... 505.092 secs...



In this case, we also add a baseline period while epoching, in order to compare and correct for brain activity **before** and **after** the trial onset.

```
In [14]: 1 for file in os.listdir("4. Epoch/Annotated files"):
2         if "raw" in file and file != '.ipynb_checkpoints__annot_raw.fif':
3             epoching(file, "4. Epoch/Annotated files")
```

Opening raw data file 4. Epoch/Annotated files/eeg_data_annot_raw.fif...

Range : 0 ... 126273 = 0.000 ... 505.092 secs

Ready.

```
[[ 0 0 3]
 [250 0 2]
 [500 0 2]
 [750 0 1]
 [1000 0 1]
 [1250 0 1]
 [1500 0 2]
 [1750 0 2]
 [2000 0 1]
 [2250 0 2]
 [2500 0 2]
 [2750 0 2]
 [3000 0 1]
 [3250 0 2]
 [3500 0 1]
 [3750 0 1]
 [4000 0 1]
 [4250 0 1]
 [4500 0 2]
 [4750 0 2]
 [5000 0 2]
 [5250 0 1]
 [5500 0 2]
 [5750 0 1]]
```

{'Possible loss': 1, 'Possible win': 2, 'timeout': 3}

Not setting metadata

23 matching events found

Applying baseline correction (mode: mean)

0 projection items activated

Loading data for 23 events and 289 original time points ...

0 bad epochs dropped

Overwriting existing file.

Load the two files you epoched to make sure the time ranges are correct: the files should report: t = -152.00...1000ms >> from -0.152 to 1 seconds.

Replace the '...' with the name of the epoched file

```
In [15]: 1 epoch_one = mne.read_epochs("4. Epoch/eeg_data_annot_raw._epo.fif", preload=True)
```

Reading C:\Users\IamAh\Desktop\UNIVERSITY - CSAI\Cognitive Neuroscience\Practical_06\Decision Making practical\4. Epoch\eeg_data_annot_raw._epo.fif ...

Found the data of interest:

t = -152.00 ... 1000.00 ms

0 CTF compensation matrices available

Not setting metadata

23 matching events found

No baseline correction applied

0 projection items activated

Same for this epoch:

```
In [16]: 1 epoch_one.get_data().shape
```

(23, 8, 289)

Step 4: Event-related potential analysis (ERP)

The goal of ERP analysis is to plot across-trials averaged signal(s), where each plotted signal is the average of all the trials belonging to one specific class.

To do so, the first step is to create two **evoked** objects (evoked objects are objects that result from the averaging across specified trials within an epoch) for each epoched file; each evoked file will extract and average across one of the two trial classes of interest (**possible win/possible loss**).

```
In [18]: 1 pw = epoch_one['Possible loss'].average()
2 pl = epoch_one['Possible win'].average()
```

In the next step, we create a dictionary where the key is the trial type and the value is a list containing the corresponding evoked objects, for each epoched file.

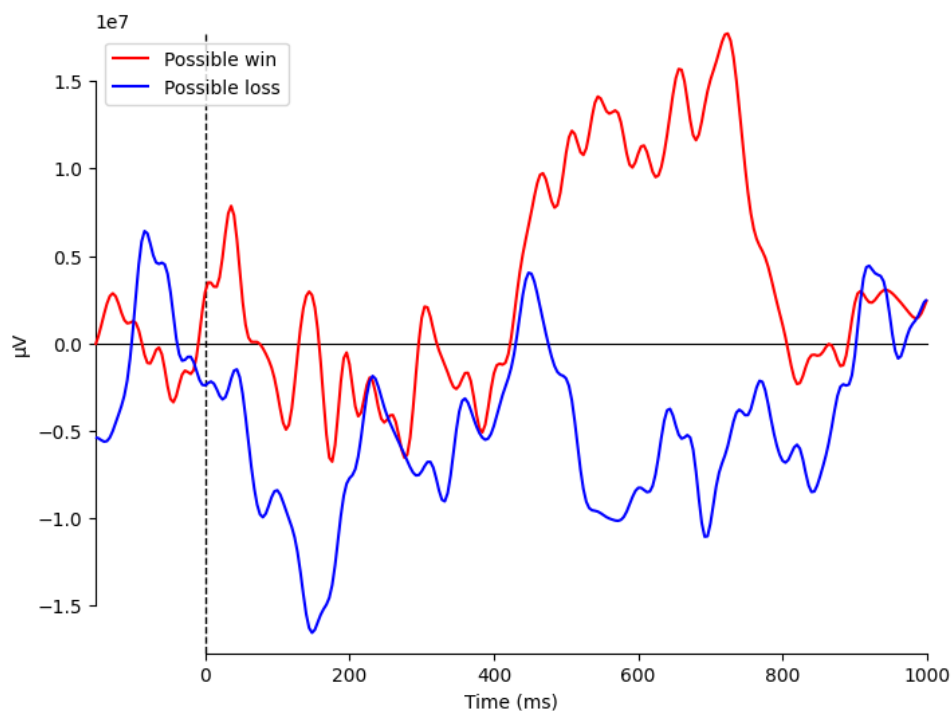
```
In [19]: 1 evokeds = {"Possible win": [pw], "Possible loss": [pl] }
```

Lastly, we plot the two ERP lines using a MNE function, to which we pass the following parameters:

1. the "evokeds" dictionary
2. the method we want to use to combine the evoked objects
3. a color dictionary (color_dict) for the line plotting
4. a linestyle (linestyle_dict)
5. the time unit we want to visualise (ms are the standard for ERP plots)

```
In [20]: 1 color_dict = {"Possible win": 'red', "Possible loss": 'blue'}
2 linestyle_dict = {'Possible win': '-', 'Possible loss': '-'}
3 mne.viz.plot_compare_evokeds(evokeds, combine = "mean", colors=color_dict,
4                               linestyles=linestyle_dict, time_unit = "ms")
```

combining channels using "mean"
combining channels using "mean"



[<Figure size 800x600 with 1 Axes>]

Step 5: Open question (MANDATORY!!)

1 Observe the ERP plot that you obtained, and try to reason on possible causes for the difference between the two labels. Use the following questions to direct your answer:

2

3 1) are the two ERPs overall the same?

4 2) are there time points in which you observe a big difference between the two? If yes, what do you think the reason might be (taking in consideration what you learned in this week's theory class)? If no, why do you think they two ERPs are the same?

#####

your answer goes here

#####

1) They do not seem to be the same 2) Possible losses evoke an early peak (200ms) absent in possible wins, also behavior at the P600 seems very different. Prospect theory evidences differences in behavior due to framing: potential wins and potential losses are not computed the same way, even when the expected value is identical.

Data Analysis DOSE

Load and check the data

```
In [22]: 1 import pandas as pd
          2 import numpy as np
          3 import os
          4 import matplotlib.pyplot as plt
```

Change the names of the variables below accordingly, otherwise your files won't open.

```
In [ ]: 1
```

In [28]:

```
1 part_name = "Nin_2022_Aug_24_1223" #here you put the main part of the name of the output file from your
2 folder_path = 'C:\\Users\\ninkh\\Documents\\CSAI\\minor\\Research Internship\\DOSE PsychoPy\\data\\' #p
3
4 part_name = "Fuma_2024_May_08_1206"
5 folder_path = "../DOSE PsychoPy/data/"
6 path = folder_path+part_name+'_output.csv'
7
8 my_data = pd.read_csv(path)
9 my_data.iloc[:,0] = my_data.iloc[:,0]+1
10 my_data.rename(columns = {'Unnamed: 0' : 'Round'},inplace = True)
11 my_data.index = my_data.iloc[:,0]
12 del my_data['Round']
13 display(my_data)
```

	Win	Loss	Sure	KL_Number	Answer	Avg_Rho	SE_Rho	Avg_Lam	SE_Lam	AvgMu	SE_Mu
Round											
1	120.0	75.0	0.0	0.618767	1	1.018399	0.232236	1.147662	0.302367	1.894196	1.084929
2	240.0	0.0	120.0	0.611497	1	1.170237	0.130458	1.147662	0.213806	1.884787	0.770599
3	120.0	105.0	0.0	0.623239	1	1.189138	0.111005	0.952549	0.133165	1.869744	0.628437
4	220.0	0.0	130.0	0.628045	1	1.461103	0.042665	0.952549	0.115324	1.858640	0.547168
5	120.0	120.0	0.0	0.639054	2	1.461833	0.037138	1.072291	0.031647	1.877094	0.486206
6	20.0	20.0	0.0	0.163277	2	1.463908	0.033444	1.082585	0.027138	1.955933	0.441639
7	40.0	40.0	0.0	0.101849	2	1.465222	0.030566	1.086226	0.024973	2.001306	0.406785
8	20.0	17.5	0.0	0.103286	1	1.467018	0.028163	1.080677	0.019906	2.042803	0.375866
9	50.0	50.0	0.0	0.069821	2	1.467828	0.026474	1.082695	0.018603	2.071915	0.352633
10	60.0	60.0	0.0	0.062310	2	1.468381	0.025054	1.083960	0.017537	2.092268	0.332917
11	50.0	0.0	30.0	0.060043	1	1.470317	0.022615	1.083960	0.016721	2.104062	0.315620
12	40.0	35.0	0.0	0.054477	1	1.470617	0.021734	1.082726	0.015649	2.109624	0.301346
13	80.0	80.0	0.0	0.047873	2	1.470805	0.020934	1.083268	0.014989	2.119985	0.288471
14	50.0	43.8	0.0	0.046727	1	1.470904	0.020196	1.082923	0.014364	2.122861	0.277573
15	90.0	90.0	0.0	0.045441	2	1.471038	0.019547	1.083294	0.013846	2.130343	0.267363
16	100.0	100.0	0.0	0.044340	2	1.471137	0.018951	1.083558	0.013385	2.135905	0.258247
17	20.0	15.0	0.0	0.044006	1	1.471168	0.018393	1.083533	0.012984	2.137702	0.250287
18	60.0	52.5	0.0	0.043819	1	1.471198	0.017881	1.083442	0.012603	2.138889	0.243078
19	40.0	0.0	22.5	0.042904	1	1.471218	0.017402	1.083442	0.012267	2.139596	0.236500
20	80.0	70.0	0.0	0.042645	1	1.471224	0.016963	1.083422	0.011953	2.139892	0.230474
21	100.0	0.0	60.0	0.042460	1	1.471229	0.016555	1.083422	0.011665	2.140136	0.224888
22	90.0	78.8	0.0	0.042458	1	1.471232	0.016175	1.083414	0.011396	2.140294	0.219698
23	20.0	12.5	0.0	0.042424	1	1.471235	0.015820	1.083412	0.011146	2.140489	0.214845
24	100.0	87.5	0.0	0.042348	1	1.471236	0.015487	1.083409	0.010911	2.140556	0.210313
25	20.0	25.0	0.0	0.042347	2	1.471239	0.015175	1.083413	0.010690	2.140710	0.206046
26	20.0	10.0	0.0	0.042266	1	1.471239	0.014880	1.083412	0.010483	2.140738	0.202041
27	70.0	0.0	40.0	0.042254	1	1.471240	0.014602	1.083412	0.010287	2.140752	0.198263
28	20.0	8.8	0.0	0.042252	1	1.471240	0.014339	1.083412	0.010101	2.140763	0.194689
29	40.0	30.0	0.0	0.042247	1	1.471240	0.014090	1.083412	0.009926	2.140769	0.191302
30	40.0	0.0	20.0	0.042247	1	1.471240	0.013853	1.083412	0.009759	2.140776	0.188086
31	20.0	7.5	0.0	0.042246	1	1.471240	0.013627	1.083412	0.009600	2.140781	0.185027
32	20.0	6.3	0.0	0.042244	1	1.471240	0.013413	1.083412	0.009449	2.140783	0.182113
33	20.0	3.8	0.0	0.042242	1	1.471240	0.013208	1.083412	0.009305	2.140783	0.179332
34	20.0	30.0	0.0	0.042242	2	1.471240	0.013012	1.083412	0.009167	2.140784	0.176675
35	50.0	37.5	0.0	0.042242	1	1.471240	0.012825	1.083412	0.009035	2.140784	0.174133
36	40.0	25.0	0.0	0.042242	1	1.471240	0.012646	1.083412	0.008908	2.140784	0.171698
37	60.0	45.0	0.0	0.042242	1	1.471240	0.012474	1.083412	0.008787	2.140784	0.169361
38	40.0	50.0	0.0	0.042242	2	1.471240	0.012308	1.083412	0.008671	2.140784	0.167118
39	20.0	35.0	0.0	0.042242	2	1.471240	0.012150	1.083412	0.008559	2.140784	0.164962
40	170.0	0.0	100.0	0.042242	1	1.471240	0.011997	1.083412	0.008451	2.140784	0.162887

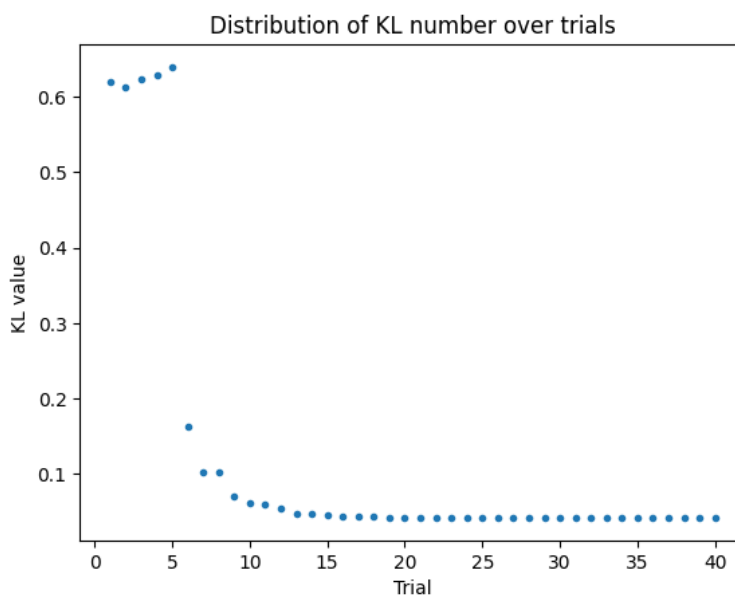
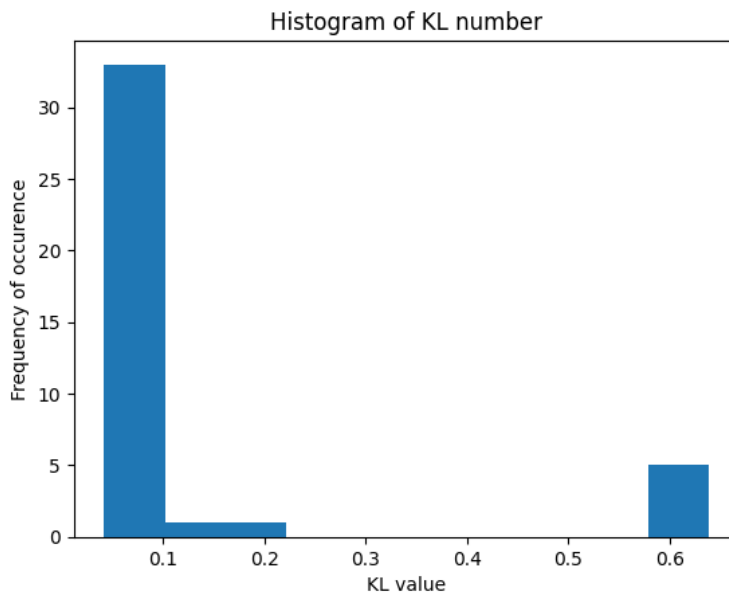
Exploratory Data Analysis

The KL number is a technical parameter that represents how much the parameters (lambda, rho and mu) could

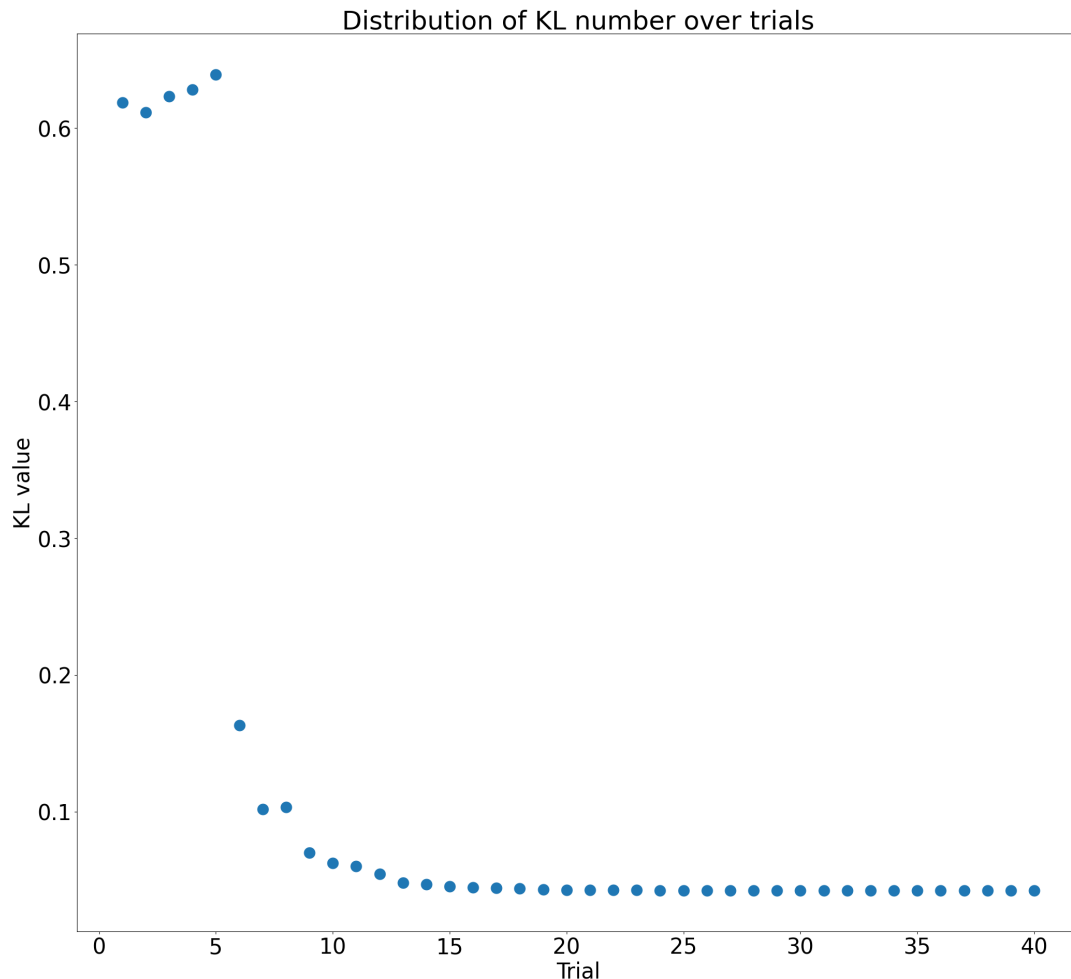
be updated if a particular question is asked (Wang, Filiba & Camerer, 2010). Every time after you made a choice to accept or to reject the gamble, the computer updated KL number for every of 138 available questions based on your choice. The computer would then choose the question with the highest KL number to be asked next, and then you would see new suggestion on the screen.

Let's see how the KL number changed throughout the experiment.

```
In [29]: 1 plt.figure()
2 plt.hist(my_data['KL_Number'])
3 plt.title("Histogram of KL number")
4 plt.xlabel("KL value")
5 plt.ylabel("Frequency of occurrence")
6 plt.show()
7
8 plt.figure()
9 plt.plot(my_data.index, my_data['KL_Number'], '.')
10 plt.title("Distribution of KL number over trials")
11 plt.xlabel("Trial")
12 plt.ylabel("KL value")
13 plt.show()
```



```
In [30]: 1 # Second plot with high resolution and custom size
2 # Set the font size
3 plt.rcParams.update({'font.size': 30})
4 plt.figure(figsize=(2000/80, 1800/80)) # Width and height in inches
5 plt.plot(my_data.index, my_data['KL_Number'], '.', markersize=30)
6 plt.title("Distribution of KL number over trials")
7 plt.xlabel("Trial")
8 plt.ylabel("KL value")
9 plt.savefig('high_resolution_plot.png', dpi=80) # Save with a custom dpi
10 plt.show()
```



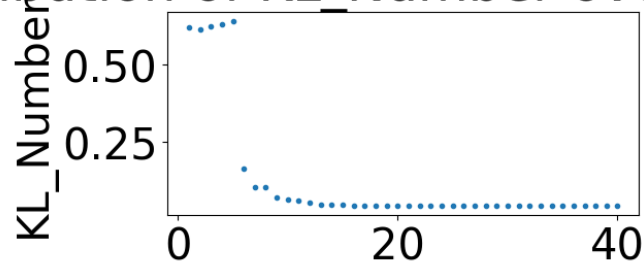
Supposedly you can see that the KL number stabilizes by the end of the experiment. When the KL number is close to zero it means that the question with the most estimated information gain was estimated to have almost zero information gain. Which means that all the other questions were estimated to have even less information gain. So asking those questions can't change the parameters much because the participant (you in this case) would most likely answer them with the same attitude she/he had when she/he answered the previous questions which determined the values of the parameters.

Now we will look into the other parameters in order to see if they are reliably estimated. These parameters are:

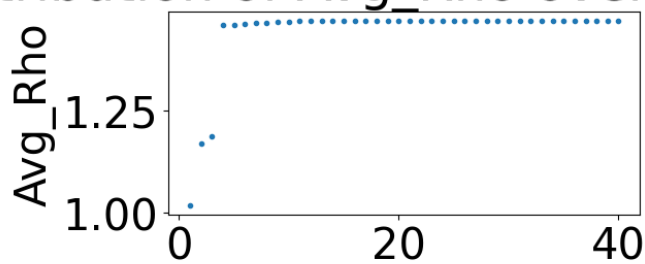
- rho for risk aversion
- lambda for loss aversion
- mu for the consistency of the choice

```
In [31]: 1 new_data = my_data.iloc[:,3:] #we only need to visualize some of the columns
2 del new_data['Answer'] #the same
3
4 fig, axes = plt.subplots(new_data.shape[1], 1, figsize=(7,25))
5 ax = axes.ravel()
6 for i in range(new_data.shape[1]):
7     ax[i].plot(new_data.index,new_data.iloc[:,i], '.')
8     ax[i].set_title("Distribution of "+new_data.columns[i]+" over the trials")
9     ax[i].set_ylabel(new_data.columns[i])
10 ax[i].set_xlabel('Trial')
11 fig.tight_layout()
```

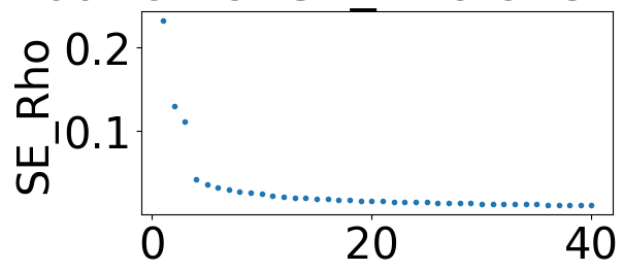
Distribution of KL_Number over the trials



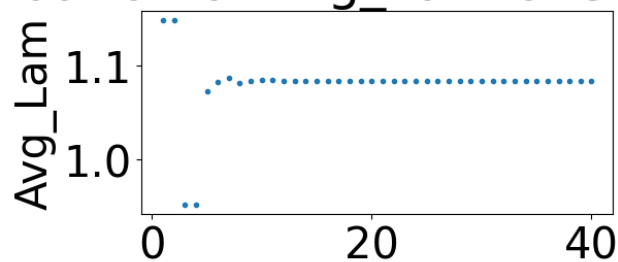
Distribution of Avg_Rho over the trials



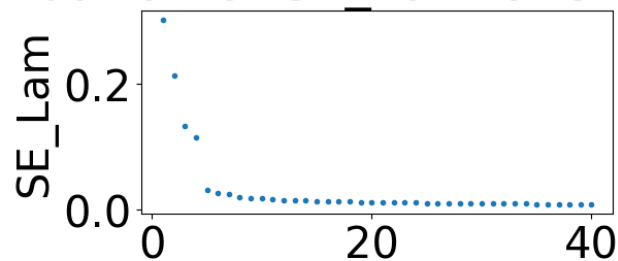
Distribution of SE_Rho over the trials



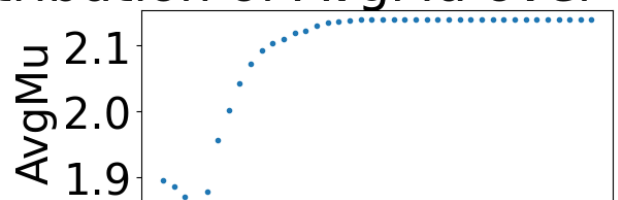
Distribution of Avg_Lam over the trials

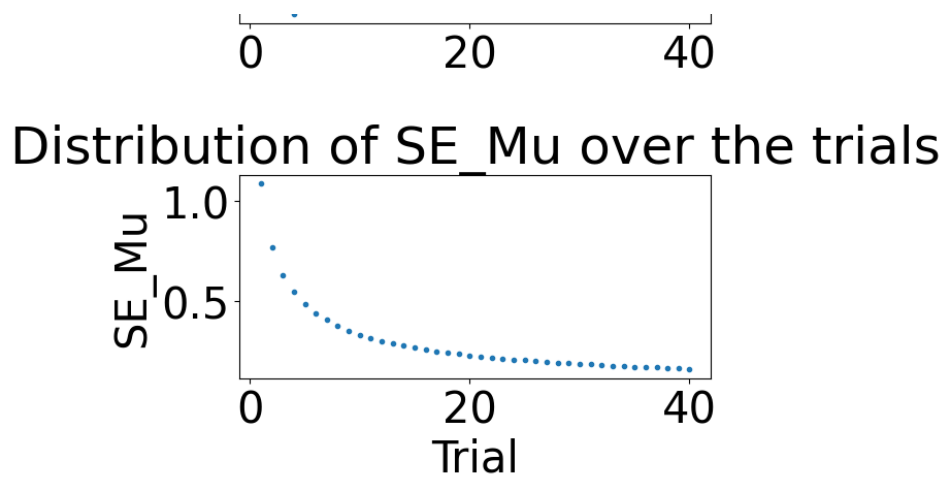


Distribution of SE_Lam over the trials



Distribution of AvgMu over the trials

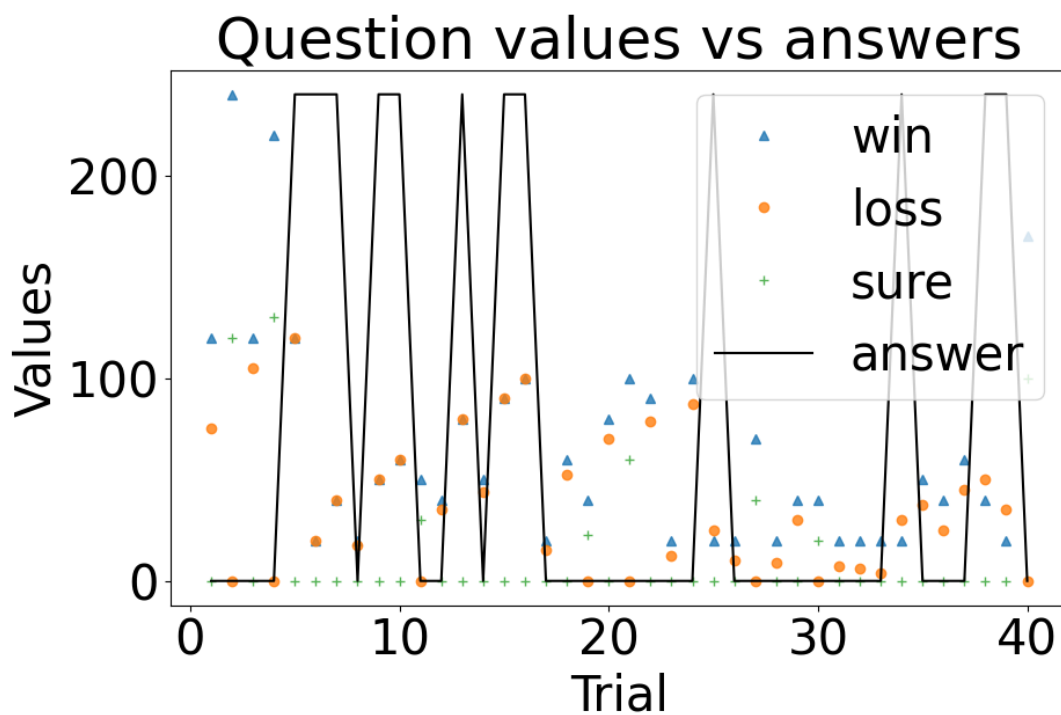




How could you describe the tendencies on the graphs? What conclusions could you make based on the distributions you see? Discuss with the group. Ask feedback from the instructor.

In order to see why particular trends occurred we could also explore the relationship between suggested win, loss and sure amounts and the answer of the participant.

```
In [32]: 1 plt.figure(figsize = (10,6))
2 plt.plot(my_data.index, my_data['Win'],'^',label='win', alpha = .8)
3 plt.plot(my_data.index, my_data['Loss'],'o',label='loss', alpha = .8)
4 plt.plot(my_data.index, my_data['Sure'],'+',label='sure', alpha = .8)
5 plt.plot(my_data.index, (my_data['Answer']-1)*np.max(my_data['Win']), 'k', label='answer')
6 plt.xlabel('Trial')
7 plt.ylabel('Values')
8 plt.title('Question values vs answers')
9 plt.legend(loc=1)
10 plt.show()
```



Describe the trends you see in the graph above (100-200 words) in the cell below. You can use the other graphs as a source of insights.

1

The values seem to be decreasing over trials, and the rate of positive answers way higher in the first half than in the second. Most answers had a "sure" value of 0. Losses were sometimes higher than wins, which seems to coincide with answers values over 200. "sure" bets (where the win was positive and the loss zero) never result in a positive value from the answer, the subject always bet in those occasions. We are not quite sure what other trends to extrapolate from the graph.

Type *Markdown* and LaTeX: α^2

References

Wang, S. W., Filiba, M., & Camerer, C. F. (2010). Dynamically optimized sequential experimentation (DOSE) for estimating economic preference parameters. Manuscript submitted for publication.